Gestion des exceptions

Marc Champesme
mailto:Marc.Champesme@lipn.univ-paris13.fr

6 avril 2014

1. Que faire en cas de problèmes ?
2. Mécanisme de gestion d'exceptions
   - Principe général
   - Différentes catégories d'exceptions
   - Traitement des exceptions
3. Gestion des exceptions en JAVA

Des problèmes peuvent survenir...

Même dans les programmes les mieux conçus, des problèmes peuvent survenir :

- panne matérielle
- mémoire insuffisante
- erreur arithmétique (division par zéro, dépassement de capacité, ...)
- violation du contrat
- ...

Que faire en cas de problème ?

Dans tous les cas il faut réagir de manière adaptée :

- Réparer puis réessayer, si possible
- Sinon, interrompre l'exécution du programme

Ce qu'il ne faut surtout pas faire :

Afficher un message d'erreur (ou ne rien faire) et poursuivre l'exécution comme si de rien n'était...

Exemple : Programme militaire de guidage de missile

Si une erreur survient lorsque votre programme calcule la position de la cible, l'utilisateur sera sûrement très content de voire s'afficher un message d'erreur sur la console une fois qu'une position "amie" aura été détruite par le missile...

Analyse : que se passe t'il quand survient un problème ?

- Le problème survient toujours lors de l'exécution d'une méthode
- En général, la conséquence première est que le contrat de cette méthode ne pourra pas être rempli
- En conséquence, la méthode ayant appelé cette méthode ne pourra probablement pas satisfaire son propre contrat. Idem pour toutes les méthodes en remontant dans la chaîne des appels jusqu'à la méthode `main()`.

Table des matières

1. Que faire en cas de problèmes ?
2. Mécanisme de gestion d'exceptions
   - Principe général
   - Différentes catégories d'exceptions
   - Traitement des exceptions
3. Gestion des exceptions en JAVA

Principe général

- Une instruction spéciale permet d'interrompre le flot d'exécution normal du programme lorsqu'un problème est détecté (instruction `throw` en JAVA).
- Un mécanisme spécifique permet au programmeur de traiter le problème et lorsque cela est possible de poursuivre l'exécution (bloc `try { ... } catch { ... }` en JAVA).
- Si le problème ne peut être traité par la méthode courante, on signale l'exception à la méthode appelante qui peut à son tour tenter de traiter le problème...
- Si ce n'est pas possible on remonte la chaîne d'appel jusqu'à la méthode main() provoquant ainsi l'interruption du programme si aucune des méthodes dans la chaîne des appels n'a pu traiter le problème de manière satisfaisante.

Table des matières

1. Que faire en cas de problèmes ?
2. Mécanisme de gestion d'exceptions
   - Principe général
   - Différentes catégories d'exceptions
   - Traitement des exceptions
3. Gestion des exceptions en JAVA

Différentes catégories d'exceptions

On distingue en général deux grandes catégories d'erreurs correspondant à deux grandes catégories d'exceptions :

Définition (erreur récupérable)

Les erreurs récupérables sont des erreurs que le logiciel peut tenter de traiter et qui peuvent permettre la poursuite de l'exécution du programme sous certaines conditions.

Définition (erreur irrécupérable)

Les erreurs irrécupérables sont des erreurs contre lesquelles, en général, le programmeur ne peut rien et qui doivent conduire à l'interruption du programme.

Erreurs récupérables

Exemple : les erreurs d'entrée sortie

Ces erreurs surviennent en général suite à un événement indépendant du programme : fichier absent, imprimante éteinte ou indisponible, ... Pour ces erreurs, une fois que l'utilisateur a connaissance du problème, il peut être en mesure d'intervenir pour résoudre le problème (par exemple : en rallumant l'imprimante...) et la reprise de l'exécution du programme peut être envisagée.

Erreurs irrécupérables

Exemple 1 : erreur interne de la machine virtuelle

Si, par manque de mémoire, la machine virtuelle ne peut plus allouer d'objets. Ce type d'erreur peut survenir à n'importe quel moment de l'exécution d'un programme.

Exemple 2 : violation du contrat

Il s'agit là d'une erreur logiciel. En général, aucune intervention au niveau logiciel ne pourra résoudre un bug... il faut corriger le bug à la source et pour cela il est important que ce bug soit signalé (par une exception) le plus tôt possible dans l'exécution du programme. Idéalement, le contrat devrait être vérifié systématiquement à chaque exécution de chaque méthode et une exception lancée dès la première violation du contrat constatée.

Table des matières

1. Que faire en cas de problèmes ?
2. Mécanisme de gestion d'exceptions
   - Principe général
   - Différentes catégories d'exceptions
   - Traitement des exceptions
3. Gestion des exceptions en JAVA

Traitement des erreurs récupérables

Ces erreurs surviennent lors de l'exécution de certaines méthodes particulières "risquées" (exemple : les méthodes qui effectuent des entrées-sorties). Il faut prévoir un traitement en deux parties :

Traitement côté fournisseur

- Signaler de manière claire dans l'interface que cette exception peut survenir.
- Lancer l'exception lorsque le problème survient.
- *Obliger* le client à gérer explicitement l'exception.

Traitement côté client

- Prévoir du code spécifique à exécuter si l'exception survient.
- Décider si l'erreur peut être gérée ou non.
- Si l'erreur est gérée le contrat doit être respecté.

Traitement des erreurs irrécupérables

Là encore, il faut prévoir un traitement en deux parties :

Traitement côté fournisseur

- Si possible (exemple : violation contrat), signaler de manière claire dans l'interface que cette exception peut survenir.
- Lancer l'exception lorsque le problème survient.
- *Laisser le client choisir* s'il gère ou non explicitement l'exception.

Traitement côté client

- Décider si l'erreur doit être gérée ou non.
- Si l'erreur est gérée, prévoir du code spécifique à exécuter si l'exception survient.
- Si l'erreur est gérée le contrat doit être respecté.

Trois catégories d'exception

En JAVA, les exceptions sont des objets dont les classes forment une hiérarchie distinguant une catégorie d'exceptions récupérables et deux catégories d'exceptions irrécupérables (cf. figure suivante).

Les exceptions en JAVA

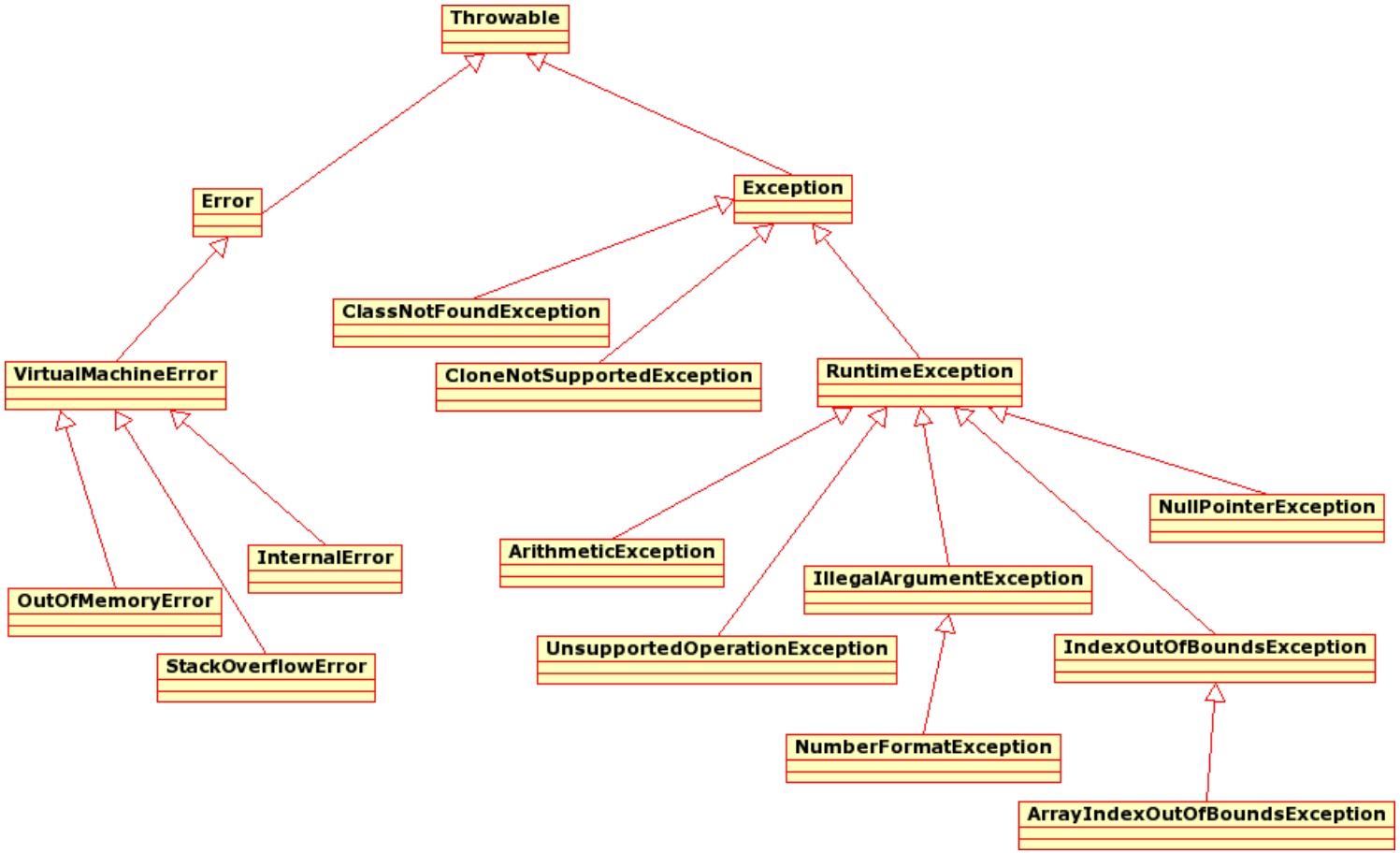

Figure : Hiérarchie des classes exceptions en JAVA

`java.lang.Error` et ses sous-classes

`java.lang.Error` et ses sous-classes (`VirtualMachineError`, `InternalError`, `OutOfMemoryError`...) sont des erreurs irrécupérables qu'il n'est pas envisageable de gérer, elles peuvent survenir à n'importe quel moment de l'exécution d'un programme sans qu'il soit possible d'en prévoir la survenue.

`java.lang.RuntimeException` et ses sous-classes

`java.lang.RuntimeException` et ses sous-classes (`IndexOutOfBoundsException`, `ArithmeticException`, `NullPointerException`...) sont elles aussi des erreurs irrécupérables correspondant à des violations du contrat. Les méthodes qui lèvent ces exceptions doivent le signaler dans leur interface (uniquement dans le commentaire javadoc) avec le tag `@throws`...

java.lang.RuntimeException exemple

```
/**
 * Constructs an empty list with the specified initial
 * capacity.
 * @param initialCapacity the initial capacity of the list
 * @throws IllegalArgumentException  if the specified initial
 *                                    capacity is negative
 */
public ArrayList(int initialCapacity) {
    if (initialCapacity < 0) {
        throw new IllegalArgumentException("The initial"
                                + "capacity must be positive");
    }
    ...
}
```

`java.lang.Exception` et ses sous-classes

`java.lang.Exception` et ses sous-classes (`IOException`, `PrinterException`...) n'héritant pas de `RuntimeException`, représentent les erreurs récupérables qui doivent être signalées dans l'interface des méthodes qui les lancent dans les commentaires javadoc (i.e. tag `@throws`) et par une clause `throws` dans l'entête de la méthode...

java.lang.Exception exemple

Classe BufferedReader :

```
/**
 * Reads a line of text. ...
 *
 * @return A String containing the contents of the line ...
 * @throws IOException If an I/O error occurs
 */
public String readLine()
                throws IOException {
    ...
        throw new IOException("...");
    ...
}
```

java.lang.Exception exemple (suite)

Utilisation de la méthode readLine :

```
public String lireLigneAvec3Essais(BufferedReader clavier) {
    int nbEssais = 0; boolean succes;
    String ligne = null;
    do { succes = true;
        try {
            ligne = clavier.readLine();
        } catch (IOException e) {
            succes = false;
        }
        nbEssais++;
    } while (!succes && nbEssais <= 3);
    if (succes) return ligne;
    else return null;
}
```

java.lang.Exception exemple (suite)

Autre utilisation possible de la méthode readLine :

```
public String lireLigneAvec3Essais(BufferedReader clavier)
                                    throws IOException {
    int nbEssais = 0;
    String ligne = null; IOException except;
    do { except = null;
        try {
            ligne = clavier.readLine();
        } catch (IOException e) {
            except = e;
        }
        nbEssais++;
    } while (except != null && nbEssais <= 3);
    if (except == null) return ligne;
    else throw except;
}
```

java.lang.Exception : exemple de clone()

Méthode clone (telle que définie dans la classe Object) :

```
protected Object clone()
                        throws CloneNotSupportedException {
    if (!(this instanceof Cloneable)) {
        throw new CloneNotSupportedException("...");
    }
    ...
}
```

java.lang.Exception : exemple de clone()

Redéfinition de la méthode clone dans une classe déclarée avec implements Cloneable :

```
public Object clone() {
    Object leClone = null;
    try {
        leClone = super.clone();
    } catch (CloneNotSupportedException e) {
        throw new InternalError("Erreur interne");
    }
    // Clonage des attributs de this si nécessaire
    return leClone;
}
```

java.lang.Exception : exemple de clone()

Redéfinition de la méthode clone dans une classe dont la super-classe a déjà redéfinit clone() :

```
public Object clone() {
    // try catch inutile car pas de clause throws
    // dans l'entête de la méthode redéfinie
    Object leClone = super.clone();
    // Clonage des attributs définis dans cette classe
    // si nécessaire
    return leClone;
}
```